



Tecnológico de Monterrey

Inteligencia artificial avanzada para la ciencia de datos

Reporte

Predicción de retrasos en vuelos mediante
regresión lineal y XGBoost

Mauricio Olguín Sánchez
A01711522

Agosto 2026

Índice general

1. Realizando la implementación «a manita»

- 1.1. Sobre la base de datos seleccionada
- 1.2. Proceso de ETL
 - 1.2.1. *Extraction*
 - 1.2.2. *Transforming*
 - 1.2.3. *Loading*
- 1.3. Análisis exploratorio de los datos
 - 1.3.1. Conclusiones
- 1.4. Modelo de regresión lineal sin *framework*
 - 1.4.1. Normalización de los datos
 - 1.4.2. Implementación del modelo
 - 1.4.3. Resultados del modelo

2. Modelo con *framework*: XGBoost

- 2.1. Modificaciones al ETL
- 2.2. Antecedentes publicados
- 2.3. Selección y configuración de XGBoost
 - 2.3.1. Configuración utilizada
 - 2.3.2. Significado de los hiperparámetros
 - 2.3.3. Técnicas de regularización
- 2.4. Resultados de XGBoost
 - 2.4.1. Conclusión sobre sesgo, varianza y ajuste
- 2.5. Comparación y veredicto final

Bibliografía

A. Figuras ampliadas

B. Códigos

Capítulo 1

Realizando la implementación «a manita»

1.1. Sobre la base de datos seleccionada

Después de investigar en páginas como [UC Irvine](#), [Kaggle](#) y [World Bank Open Data](#), encontré en [OpenML](#) un *dataset* que se me hizo interesante. Tras aplicar varios filtros, seleccioné la base de datos [Airlines_DepDelay_10M](#).

Esta base de datos está construida a partir de datos provenientes de la Oficina de Estadísticas de Transporte de los Estados Unidos (*Bureau of Transportation Statistics*, BTS), específicamente, de la base de datos *Airline On-Time Performance Data*. La base de datos seleccionada contiene 10,000,000 de registros y 10 variables, y corresponde a un problema de regresión. A continuación, presento una breve descripción general del *dataset*:

Propósito: Predecir el tiempo de retraso en la salida de los vuelos, medido en minutos (*Departure Delay*).

Fecha de los datos: Desde 1987 hasta 2013.

Formato: ARFF (*Attribute-Relation File Format*).

Tipo de tarea: Regresión.

Número de registros: 10,000,000.

Número de variables: 10.

Aunque OpenML proporciona información sobre cada variable, la fuente original del *Bureau of Transportation Statistics* ofrece una descripción más detallada de los datos:

DepDelay - numérico. Diferencia, en minutos, entre la hora de salida programada y la hora de salida real del vuelo. Los valores negativos indican que el vuelo salió antes de lo programado.

Month - numérico. Mes en el que se realizó el vuelo, con valores del 1 al 12, donde 1 corresponde a enero y 12 a diciembre.

DayofMonth - numérico. Día del mes en el que se realizó el vuelo, con valores del 1 al 31.

DayOfWeek - numérico. Día de la semana en el que se realizó el vuelo, representado mediante valores del 1 al 7.

CRSDepTime - numérico. Hora de salida programada del vuelo según el sistema de reservas computarizado de la aerolínea (CRS), expresada en hora local con formato HHMM.

CRSArrTime - numérico. Hora de llegada programada del vuelo según el sistema de reservas computarizado de la aerolínea (CRS), expresada en hora local con formato HHMM.

UniqueCarrier - categórico. Código único utilizado para identificar a la aerolínea que opera el vuelo.

Origin - categórico. Código del aeropuerto de origen del vuelo.

Dest - categórico. Código del aeropuerto de destino del vuelo.

Distance - numérico. Distancia total del vuelo, expresada en millas.

1.2. Proceso de ETL

1.2.1. *Extraction*

Como mencioné anteriormente, obtuve los datos del siguiente enlace: <https://www.openml.org/search?type=data&status=active&id=42728>

Si se quisieran obtener datos más actuales, habría que descargarlos manualmente o crear un *script* para obtenerlos desde el siguiente enlace: <https://transtats.bts.gov/PREZIP/>.

En esta página se recopilan datos desde 1987 hasta junio de 2026.

1.2.2. *Transforming*

Sobre los atributos *Month*, *DayofMonth* y *DayOfWeek*

Las variables *Month*, *DayofMonth* y *DayOfWeek* se almacenan como números, pero describen ciclos. Por ello, no conviene introducirlas directamente como cantidades lineales: diciembre y enero, o domingo y lunes, son periodos consecutivos aunque sus códigos numéricos se encuentren en extremos opuestos. En el procesamiento actual cada variable x se sustituye por un par de componentes:

$$x_{\sin} = \sin\left(\frac{2\pi x}{P}\right), \quad x_{\cos} = \cos\left(\frac{2\pi x}{P}\right),$$

donde P vale 12 para el mes, 31 para el día del mes y 7 para el día de la semana.

Esta codificación conserva la cercanía entre los extremos de cada ciclo y produce seis predictores numéricos. Se utiliza tanto en el modelo manual como en XGBoost. En el caso del día del mes se utiliza un periodo fijo de 31 días, por lo que la representación es una aproximación para los meses más cortos.

Las variables *UniqueCarrier*, *Origin* y *Dest* se excluyen únicamente del modelo manual para conservar una implementación manejable con listas y ciclos. En XGBoost sí se incorporan mediante una codificación categórica ajustada exclusivamente con los datos de entrenamiento.

Sobre los atributos *CRSDepTime* y *CRSArrTime*

Las variables *CRSDepTime* y *CRSArrTime* representan las horas programadas de salida y llegada de los vuelos. En el conjunto de datos original, estas se presentan en formato numérico HHMM (por ejemplo, las 13:20 se representan como 1320).

Sin embargo, esta representación no es adecuada para utilizarla directamente en los modelos. El formato HHMM introduce saltos numéricos que no corresponden al tiempo real. Por ejemplo, 1059 (10:59) y 1100 (11:00) están separados por un minuto, pero el modelo podría interpretar una diferencia de 41 unidades. Esto hace que los horarios parezcan más alejados de lo que realmente están.

Antes de la conversión se valida que ambos atributos representen una hora posible: deben ser enteros, contener horas entre 0 y 23 y minutos entre 0 y 59. El valor 2400 se admite como una representación especial de medianoche y se transforma en 0. Si cualquiera de los dos horarios de una observación es inválido, la observación completa se elimina. Esta revisión encontró un único registro con *CRSArrTime*=2096; al no existir evidencia para reconstruir su valor correcto, se eliminó en lugar de imputarlo.

Después de la validación, se separaron las

horas y los minutos y se aplicó la siguiente ecuación:

$$\text{Minutos}_{\text{desde la medianoche}} = 60 \times \text{Hora} + \text{Minutos}$$

Así, 1059 se convierte en 659 y 1100 en 660. La transformación elimina la discontinuidad aritmética propia del formato HHMM y coloca los horarios en el intervalo de 0 a 1439 minutos. No debe interpretarse como la duración de un vuelo: las horas son locales y la diferencia entre salida y llegada también puede atravesar la medianoche o zonas horarias distintas.

Sobre el formato del archivo de los datos

Al comenzar la exploración, cada ejecución tardaba aproximadamente 45 segundos solamente en cargar los datos. Buscando una forma de reducir este tiempo, encontré el formato **parquet**. Este formato está optimizado para trabajar con datos grandes, acelerar su lectura y utilizar mejor el almacenamiento [1].

Después de implementarlo, el archivo pasó de 385 MB a 56 MB y el tiempo de carga bajó de 45 segundos a 2 segundos o menos. Esto me permitió realizar nuevas pruebas con mayor rapidez.

1.2.3. Loading

Después de eliminar la observación inválida quedaron 9,999,999 registros. En este proyecto quiero evaluar vuelos de la misma población histórica y no simular la predicción de un periodo futuro. Además, el *dataset* no contiene el año ni una fecha completa que permita hacer una separación cronológica correcta. Por estas razones, mezclé los datos con la semilla 42 antes de dividirlos. La división conserva 98 % de los datos para *train*, 1 % para *validation* y 1 % para *test*: 9,799,999, 99,999 y 100,001 observaciones, respectivamente. La diferencia de una observación se debe al redondeo de las propor-

ciones enteras. Los tres conjuntos son disjuntos. [7]

En las pruebas parciales, la fracción de muestreo se aplica solamente a *train*. Los conjuntos de *validation* y *test* permanecen completos para que sus métricas sean estables y comparables entre ejecuciones. *Train* se utiliza para ajustar los parámetros; *validation*, para comparar configuraciones; y *test* queda reservado para una sola evaluación una vez elegido el modelo y congelados sus hiperparámetros.

1.3. Análisis exploratorio de los datos

Para tener un primer acercamiento general a los datos realicé un gráfico de pares o *pairplot*, que se muestra en la figura 1.1.

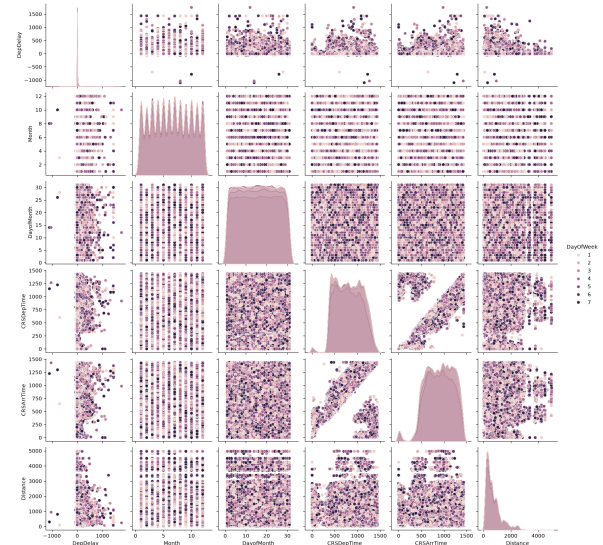
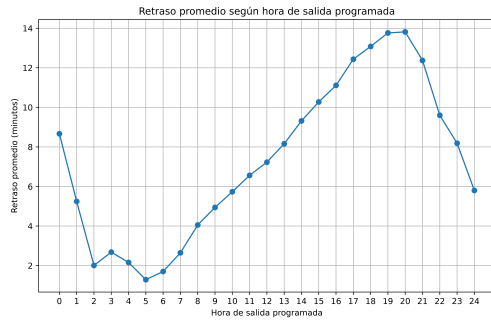


Figura 1.1: *Pairplot* utilizado para comparar las variables. Para reducir el tiempo de cómputo se graficó un millón de observaciones y cada punto se identificó según el día de la semana.

En estas gráficas podemos observar una zona en la que se concentra la mayoría de los valores de *DepDelay*. También aparecen valores atípicos negativos, los cuales representan vuelos que salieron antes de la hora programada. Por otro lado, *CRSDepTime* y *CRSArrTime* presentan agrupaciones claras porque ambas variables representan

horarios.

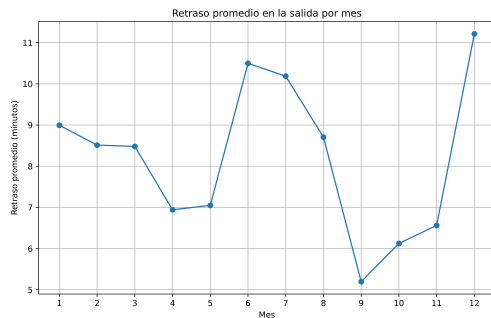
Después de observar el panorama general, realicé gráficas más específicas. La figura 1.2 muestra el retraso promedio según la hora, el día de la semana y el mes.



(a) Promedios de retrasos por cada hora.



(b) Promedios de retrasos por cada día de la semana.



(c) Promedios de retrasos por cada mes.

Figura 1.2: Promedios de retraso según distintos periodos de tiempo.

De estas gráficas, puedo destacar lo siguiente:

- Por hora, el retraso promedio es menor alrededor de las 5 de la mañana. Después aumenta poco a poco hasta alcanzar su punto más alto cerca de las 7 de la tarde.

- Por día de la semana, el retraso promedio aumenta conforme se acerca el fin de semana y alcanza su punto más alto el viernes.

- Por mes, los retrasos aumentan durante el verano y al inicio del año. Esto coincide con periodos vacacionales, aunque con estos datos no puedo afirmar que esa sea la causa.

Estos promedios pueden verse afectados por los valores atípicos. Para observar mejor cómo se distribuyen los retrasos, realicé el histograma de la figura 1.3.

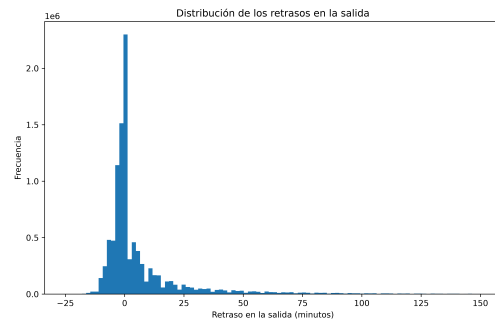


Figura 1.3: Histograma de los retrasos. La escala se recortó de -25 a 150 minutos porque los pocos valores atípicos reducen la visibilidad de la mayoría de los datos.

La mayoría de los vuelos se concentra cerca de cero minutos de retraso. También existen valores negativos, que corresponden a vuelos que salieron antes de la hora programada.

Por último, utilicé la matriz de correlación de la figura 1.4 para revisar la relación lineal entre las variables.

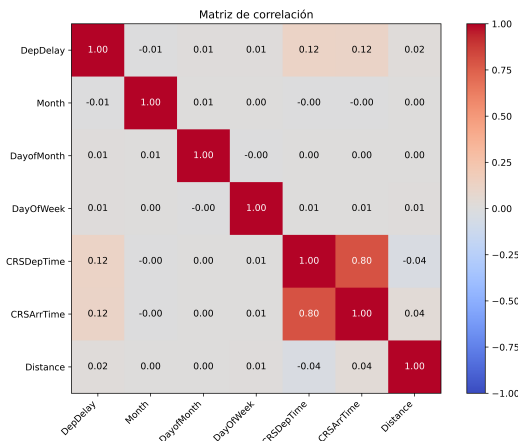


Figura 1.4: Matriz de correlación.

Como se puede observar, la mayoría de las variables tiene una correlación lineal débil con *DepDelay*. *CRSDepTime* y *CRSArrTime* tienen los valores positivos más altos, de 0.12 en ambos casos, pero la relación sigue siendo pequeña. También existe una correlación de 0.80 entre estas dos variables, algo esperado porque representan los horarios programados de un mismo vuelo.

1.3.1. Conclusiones

A partir de este análisis, puedo concluir lo siguiente:

Naturaleza de los datos: La baja correlación con *DepDelay* indica que una regresión lineal probablemente explicará sólo una pequeña parte de los retrasos.

Multicolinealidad temporal: La relación entre las horas de salida y llegada muestra que ambas aportan información parecida, lo que puede afectar la forma en la que el modelo asigna sus pesos.

Diferencia de escalas: Las variables utilizan escalas muy distintas. Por ejemplo, la distancia puede contener miles de millas, mientras que las demás variables tienen valores mucho menores.

Para resolver la diferencia de escalas, normalicé los datos antes de entrenar el modelo

manual. Esto evita que las variables con valores grandes dominen el cálculo del costo y ayuda a que el Descenso de Gradiente encuentre los pesos de una forma más rápida y estable. En la siguiente sección explico cómo realicé esta implementación «a manita».

1.4. Modelo de regresión lineal sin *framework*

1.4.1. Normalización de los datos

Para normalizar los datos utilicé el método *Min-Max*:

$$X_{\text{norm}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Esta transformación coloca las variables utilizadas durante el ajuste dentro de una escala común y facilita la convergencia del descenso de gradiente.

Los valores mínimo y máximo se calculan únicamente con *train*; posteriormente, esos mismos parámetros se aplican a *validation* y *test*. De esta forma se evita que información de los conjuntos de evaluación intervenga en la preparación del modelo. En la implementación manual se normalizan sus nueve predictores numéricos: *CRSDepTime*, *CRSArrTime*, *Distance* y los seis componentes seno y coseno de las variables de calendario. Si una columna de entrenamiento es constante, se sustituye por cero en los tres conjuntos para evitar una división entre cero.

1.4.2. Implementación del modelo

Una vez preparados y normalizados los datos, implementé el modelo de regresión lineal a partir de las variables predictoras. Para una observación con n características, la predicción se expresa como:

$$\hat{y} = b + \sum_{j=1}^n w_j x_j \quad (1.1)$$

donde $\hat{y}$ representa la predicción del modelo, b el bias, w_j el peso asociado a la característica x_j , y n el número total de variables predictoras. A continuación se describen las funciones que componen esta implementación.

Función de hipótesis y cálculo del error

La función de hipótesis recibe las características de una observación junto con los pesos y el bias actuales, y devuelve una predicción:

$$h(x) = b + \sum_{j=1}^n w_j x_j \quad (1.2)$$

Sobre esta predicción se calcula el error de cada observación como la diferencia entre el valor predicho y el real:

$$e_i = \hat{y}_i - y_i \quad (1.3)$$

Tanto la hipótesis como el error son la base del resto de la implementación, ya que el cálculo del costo y de los gradientes dependen directamente de ellos.

Función de costo

Para medir qué tan alejadas se encuentran las predicciones de los valores reales utilicé el *Mean Squared Error* (MSE):

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m (\hat{y}_i - y_i)^2 \quad (1.4)$$

donde m representa el número de observaciones utilizadas. Elevar los errores al cuadrado evita que los errores positivos y negativos se cancelen entre sí, y penaliza en mayor medida los errores de mayor magnitud. El objetivo del Descenso de Gradiente es encontrar los valores de w y b que minimicen esta función.

Gradientes de los parámetros

Para actualizar los parámetros del modelo es necesario conocer cómo cambia la función de costo respecto a cada uno de ellos. Las derivadas parciales del costo respecto a un peso w_j y respecto al bias son:

$$\frac{\partial J}{\partial w_j} = \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) x_{ij} \quad (1.5)$$

$$\frac{\partial J}{\partial b} = \frac{2}{m} \sum_{i=1}^m (\hat{y}_i - y_i) \quad (1.6)$$

Ambas expresiones se calculan recorriendo todas las observaciones del lote y acumulando el error correspondiente; la única diferencia es que, en el caso del bias, el error no se multiplica por ninguna característica, ya que la derivada de la hipótesis respecto a b es igual a 1. El resultado de cada gradiente indica la dirección y magnitud en la que debe modificarse el parámetro correspondiente para disminuir el costo.

Actualización de los parámetros

Con los gradientes calculados, los parámetros se actualizan mediante el algoritmo de Descenso de Gradiente:

$$w_j^{\text{nuevo}} = w_j - \alpha \frac{\partial J}{\partial w_j} \quad (1.7)$$

$$b^{\text{nuevo}} = b - \alpha \frac{\partial J}{\partial b} \quad (1.8)$$

donde α representa la tasa de aprendizaje (*learning rate*). El signo negativo es fundamental, ya que el gradiente indica la dirección de mayor incremento del costo; por lo tanto, se avanza en la dirección contraria para acercarse a un mínimo de la función.

Entrenamiento por mini-batch

Debido a que el conjunto completo de entrenamiento contiene aproximadamente 9.8 millones de observaciones, calcular los gradientes utilizando todos los datos antes de

cada actualización resultaría costoso. Por ello utilicé *Mini-Batch Gradient Descent* y dividí las observaciones en lotes de 2048. Esto me permitió actualizar los parámetros varias veces por época y mantener explícitos los ciclos y las listas de la implementación manual.

Durante cada época recorro todos los lotes y repito los siguientes pasos:

1. Realizo una predicción.
2. Calculo el error.
3. Calculo el costo.
4. Calculo los gradientes.
5. Actualizo w y b .
6. Al terminar la época, calculo el costo promedio de todos los lotes.

Uso de los conjuntos de entrenamiento, validación y prueba

El conjunto de *train* es el único utilizado para actualizar los pesos y el bias en cada época. El conjunto de *validation*, en cambio, no participa en esta actualización: después de cada época se usan los parámetros obtenidos hasta ese momento para generar predicciones sobre él y calcular su costo, lo que permite observar qué tanto generaliza el modelo a datos no usados para ajustarlo. El conjunto de *test* se mantiene completamente al margen durante el entrenamiento y solo se utiliza al final, con los parámetros ya definidos, para obtener una evaluación final e independiente del desempeño del modelo.

1.4.3. Resultados del modelo

Para la ejecución final del modelo manual utilicé todos los datos de *train*, es decir, 9,799,999 observaciones. Para *validation* utilicé 99,999 observaciones y para *test*, 100,001. Los parámetros que utilicé fueron los siguientes:

- $learning\ rate = 0,01$

- $epochs = 20$
- $batch\ size = 2048$
- $random\ state = 42$

La semilla 42 permite que los pesos iniciales y el orden de los datos sean iguales cada vez que se ejecuta el programa. La curva de la figura 1.5 muestra cómo cambia el error mientras el modelo aprende. Sin embargo, el valor de *train* que aparece en la curva se calcula mientras se actualizan los pesos, por lo que no es exactamente igual al MSE final. Las métricas de la tabla sí se calcularon al final, cuando el entrenamiento ya había terminado.

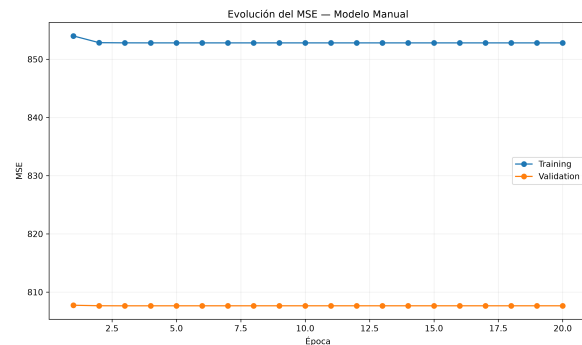


Figura 1.5: Evolución del MSE de entrenamiento y validación del modelo manual durante las 20 épocas.

Las métricas obtenidas con la partición aleatoria actual fueron:

Conjunto	MAE	RMSE	R^2	Error medio
<i>Train</i>	14.1480	29.2039	0.0172	-0,2757
<i>Validation</i>	14.1094	28.4193	0.0191	-0,2473
<i>Test</i>	14.0481	29.1341	0.0181	-0,2175

Cuadro 1.1: Resultados definitivos del modelo manual. El error medio se define como predicción menos valor real y se expresa en minutos.

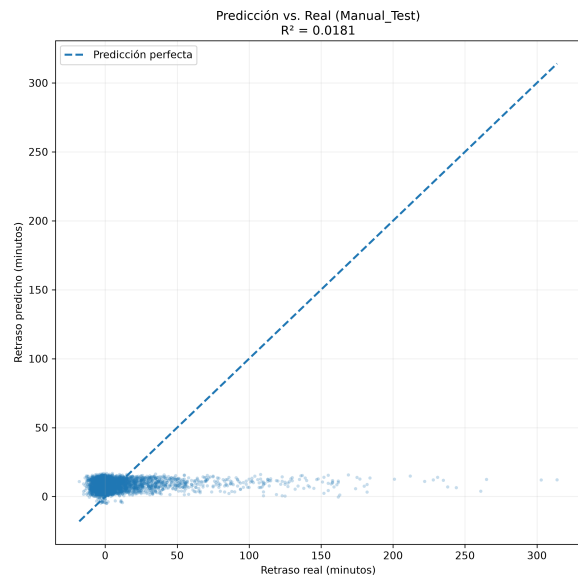
Podemos observar que el RMSE de *train* es mayor que el de *validation*. Esto no significa que el modelo funcione mejor con datos nuevos. La razón es que cada conjunto tiene retrasos extremos diferentes y el RMSE es muy sensible a estos valores. Al comparar *validation* con *test*, el RMSE aumenta

2.51 %, el MAE disminuye 0.43 % y R^2 sólo cambia -0.0010 . Como los cambios son pequeños, podemos decir que el modelo se comporta de forma parecida en ambos conjuntos.

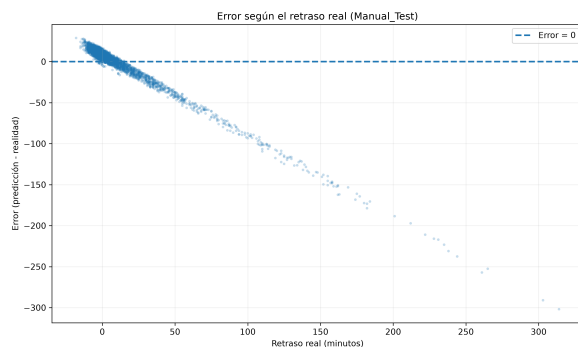
El error absoluto mediano fue de 9.1324 minutos en *validation* y de 9.1173 en *test*. En ambos conjuntos, aproximadamente 23.7 % de las predicciones quedó dentro de ± 5 minutos, 55.3 % dentro de ± 10 , 79.6 % dentro de ± 15 y 92.9 % dentro de ± 30 . Esto quiere decir que el modelo funciona mejor cuando permitimos un margen amplio. La diferencia entre MAE y RMSE también muestra que existen algunos errores muy grandes.

La figura 1.6 contiene dos gráficas sencillas. En la primera, cada punto representa un vuelo: en el eje horizontal aparece su retraso real y en el vertical aparece la predicción. Mientras más cerca se encuentre un punto de la línea punteada, mejor fue la predicción. La segunda gráfica muestra el error de cada vuelo. La línea horizontal representa un error igual a cero; los puntos que están debajo de ella son retrasos que el modelo subestimó.

Estas gráficas sirven para encontrar errores que no se distinguen fácilmente en una tabla. En este caso, muestran que la mayoría de las predicciones se concentra en un rango pequeño y que el modelo predice valores mucho menores cuando el retraso real es muy grande.



(a) Retraso predicho contra retraso real.



(b) Error según el retraso real.

Figura 1.6: Diagnóstico del modelo manual sobre el conjunto de prueba.

Conclusión sobre sesgo, varianza y ajuste

El modelo tiene un **sesgo alto**, ya que su R^2 se mantiene entre 0.0172 y 0.0191. Esto significa que explica menos de 2 % de los cambios que existen en los retrasos. También tiene una **varianza baja**, porque los resultados de *train*, *validation* y *test* son muy parecidos. Por estas dos razones, podemos concluir que el modelo tiene **underfitting**. En otras palabras, el modelo es demasiado sencillo para explicar correctamente el problema. No se observa *overfitting*, pues el desempeño no empeora de forma importante al utilizar datos que no se usaron para entrenarlo.

Es importante no confundir este sesgo con el error medio de la tabla. El error medio es simplemente el promedio de la diferencia entre la predicción y el valor real. En *test* fue de $-0,2175$ minutos, por lo que el modelo subestima los retrasos por una cantidad muy pequeña en promedio. Aun así, el sesgo del modelo sigue siendo alto porque su R^2 es muy bajo.

Debido a este *underfitting*, en el siguiente capítulo pruebo XGBoost. Este modelo puede encontrar relaciones más complejas y también permite utilizar las variables categóricas que descarté en la implementación manual.

Capítulo 2

Modelo con *framework*: XGBoost

2.1. Modificaciones al ETL 2.2. Antecedentes publicados

XGBoost parte del mismo ETL que expliqué en el primer capítulo. Para no repetir esa información, en esta sección sólo explico los cambios que hice específicamente para este modelo.

A diferencia del modelo manual, para XGBoost sí conservé *UniqueCarrier*, *Origin* y *Dest*. Estas variables pueden ayudar al modelo a encontrar diferencias entre aerolíneas, aeropuertos y rutas. Como son variables categóricas, les apliqué *One-Hot Encoding*. Esta transformación solamente se aplicó a esas tres columnas; las variables de calendario conservaron la codificación cíclica explicada anteriormente.

El codificador se ajustó únicamente con *train*. Después utilicé ese mismo codificador para transformar *validation* y *test*, evitando que estos conjuntos aportaran información durante el entrenamiento. Al terminar este proceso se obtuvo una matriz con **776 predictores**. La matriz se guardó de forma dispersa y con números de 32 bits para reducir el consumo de memoria.

Por último, para XGBoost no utilicé la normalización Min-Max que se aplicó al modelo manual. Al estar formado por árboles, XGBoost realiza divisiones sobre los valores de cada variable y no necesita que todas tengan la misma escala.

Antes de seleccionar XGBoost revisé cinco artículos relacionados con la predicción de retrasos. Sus resultados muestran que los modelos basados en árboles pueden funcionar bien para este problema. Sin embargo, cada trabajo utiliza datos y objetivos diferentes, por lo que no sería correcto comparar directamente sus métricas con las mías. Estos artículos utilizan información del clima, la congestión, la capacidad de los aeropuertos, las operaciones o la propagación de retrasos. Algunos también eliminan valores extremos o trabajan con un solo aeropuerto. Mi *dataset* contiene principalmente información programada del vuelo. Por esta razón, sus resultados no son directamente comparables con los míos: ellos cuentan con variables que explican mejor por qué ocurre un retraso.

2.3. Selección y configuración de XGBoost

Elegí XGBoost porque construye árboles de forma secuencial y cada árbol intenta corregir parte de los errores del anterior. Gracias a esto puede encontrar relaciones no lineales entre los horarios, las fechas, la distancia, la aerolínea y la ruta. Esto es útil porque en el análisis exploratorio observé que la relación lineal con *DepDelay* era muy débil. Además, Manna et al. obtuvieron buenos resultados

Trabajo	Resultado principal	Diferencia con mi trabajo
Kalliguddi y Leboullec [3]	Regresión: $R^2 = 0,84$ y RMSE de 21.2 min. Random Forest: $R^2 = 0,94$ y RMSE de 12.5 min.	Utiliza variables operacionales que no tengo disponibles.
Rebollo y Balakrishnan [6]	Cerca de 19 % de error de clasificación y 21 min de error mediano a dos horas.	Predice retrasos de rutas y no de vuelos individuales.
Li y Jing [4]	$R^2 = 0,90$, MAE de 5.19 y RMSE de 12.42 min; 93.92 % dentro de ± 15 min.	Utiliza datos de congestión, capacidad y red.
Guo et al. [2]	RFR-MIC funcionó mejor que los modelos comparados. Utilizar varias rutas también mejoró el resultado.	Utiliza información de varias rutas. El resumen no muestra métricas numéricas.
Manna et al. [5]	GBDT obtuvo un R^2 de 92.32 % para llegadas y 94.85 % para salidas.	Es el modelo más parecido a XGBoost, pero su evaluación fue diferente.

Cuadro 2.1: Resultados de los cinco antecedentes revisados. Las métricas pertenecen a conjuntos y objetivos diferentes.

con Gradient Boosted Decision Tree [5], que pertenece a la misma familia de modelos.

2.3.1. Configuración utilizada

La configuración utilizada para entrenar XGBoost fue la siguiente:

- $n_estimators = 500$
- $max_depth = 6$
- $learning_rate = 0.05$
- $subsample = 0.8$
- $colsample_bytree = 0.8$
- $min_child_weight = 5$
- $reg_lambda = 1.0$
- $tree_method = hist$
- $max_bin = 256$
- $eval_metric = rmse$
- $objective = reg:squarederror$
- $n_jobs = -1$
- $random_state = 42$

El modelo recorrió los 500 árboles. No utilicé detención temprana porque en las pruebas anteriores nunca llegó a detener el entrenamiento y, por lo tanto, no cambió el resultado.

2.3.2. Significado de los hiperparámetros

Cada opción controla una parte diferente del entrenamiento:

$n_estimators$: Es la cantidad de árboles que construye el modelo. En este caso se utilizan 500 árboles.

max_depth : Indica la profundidad máxima de cada árbol. Una profundidad de 6 permite encontrar relaciones más complejas sin hacer árboles demasiado grandes.

$learning_rate$: Controla cuánto aporta cada árbol nuevo a la predicción final. El valor 0.05 hace que el modelo aprenda poco a poco.

$subsample$: Define qué porcentaje de las observaciones utiliza cada árbol. Con 0.8, cada árbol utiliza 80 % de los datos de entrenamiento.

$colsample_bytree$: Define qué porcentaje de los predictores puede utilizar cada árbol. El valor 0.8 significa que utiliza 80 % de las columnas.

min_child_weight : Exige un peso mínimo de 5 para crear una nueva rama. Esto evita divisiones basadas en muy pocos datos.

reg_lambda: Aplica regularización L2 a los valores de las hojas. Sirve para limitar predicciones demasiado extremas.

tree_method: Indica cómo se buscan las divisiones de los árboles. El método *hist* agrupa los valores en intervalos para reducir el tiempo y el uso de memoria.

max_bin: Establece hasta 256 intervalos para el método *hist*. Más intervalos permiten divisiones más detalladas, pero también consumen más recursos.

eval_metric: Define la métrica que se observa durante el entrenamiento. Se utilizó RMSE porque penaliza más los errores grandes.

objective: Indica el tipo de problema y la función que XGBoost intenta minimizar. *reg:squarederror* corresponde a una regresión basada en el error cuadrático.

n_jobs: Controla cuántos núcleos del procesador puede utilizar el entrenamiento. El valor -1 permite utilizar todos los núcleos disponibles.

random_state: Fija la semilla en 42 para que los pasos aleatorios del entrenamiento puedan repetirse.

2.3.3. Técnicas de regularización

Para evitar que XGBoost se ajuste demasiado a *train*, utilicé varias formas de regularización. La profundidad máxima de 6 y *min_child_weight* de 5 limitan qué tan complejos pueden ser los árboles. *Subsample* hace que cada árbol utilice 80 % de las observaciones y *colsample_bytree* hace que utilice 80 % de las columnas. El *learning rate* de 0.05 evita que cada árbol cambie demasiado la predicción. Por último, la regularización L2 limita los valores de las hojas. Con estos parámetros se busca disminuir la varianza y evitar el *overfitting*.

2.4. Resultados de XGBoost

Al igual que con el modelo manual, evalué XGBoost con los conjuntos completos de *train*, *validation* y *test*. Las métricas obtenidas fueron las siguientes:

Conjunto	MAE	RMSE	R^2	Error medio
<i>Train</i>	13.7983	28.6638	0.0532	0.0003
<i>Validation</i>	13.7753	27.9544	0.0509	0.0319
<i>Test</i>	13.7230	28.6896	0.0479	0.0567

Cuadro 2.2: Resultados definitivos de XGBoost. El error medio se expresa en minutos.

Al comparar *validation* con *test*, el RMSE aumenta 2.63 %, el MAE disminuye 0.38 % y R^2 sólo cambia -0.0031 . El error absoluto mediano fue de 8.4836 minutos en *validation* y de 8.5006 en *test*. Como estos valores son muy parecidos, podemos decir que el modelo mantiene su desempeño al utilizar datos nuevos.

En *test*, 27.53 % de las predicciones quedó dentro de ± 5 minutos, 58.50 % dentro de ± 10 , 78.11 % dentro de ± 15 y 92.94 % dentro de ± 30 . Al igual que en el modelo manual, el porcentaje mejora bastante cuando se permite un margen más amplio.

En la figura 2.1 podemos observar cómo disminuye el RMSE conforme se agregan árboles. Las dos líneas continúan bajando hasta el árbol 500 y no se separan cada vez más. Esto apoya la idea de que el modelo no presenta *overfitting* durante estas rondas.

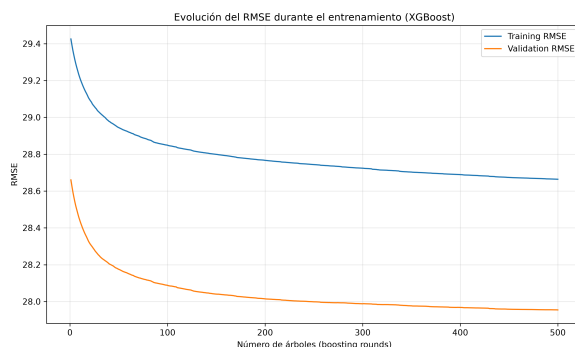
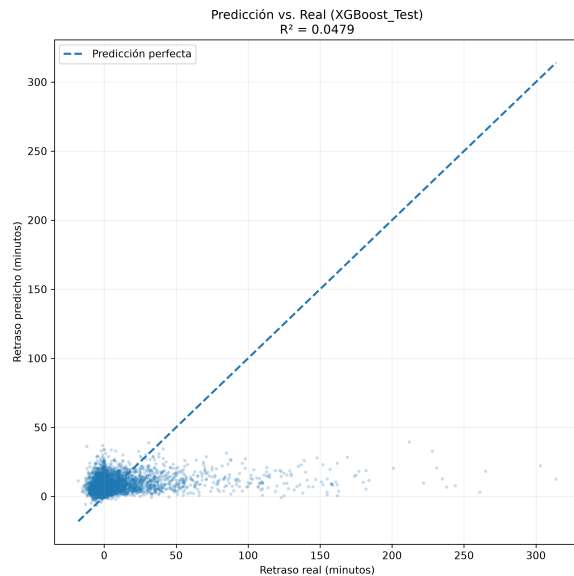
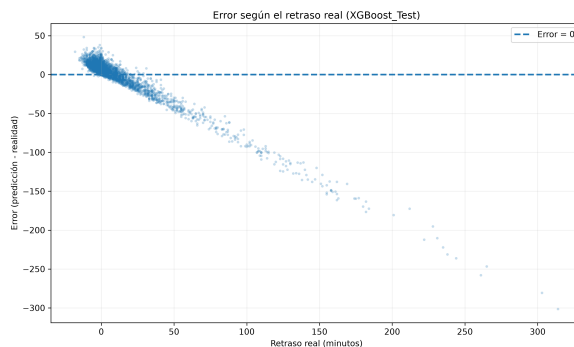


Figura 2.1: Evolución del RMSE durante los 500 árboles de XGBoost. El error de entrenamiento y el de validación continúan bajando.

La figura 2.2 se interpreta de la misma forma que la del modelo manual. La primera gráfica permite revisar qué tan cerca se encuentra cada predicción del valor real. La segunda muestra si el modelo se equivoca por arriba o por debajo del retraso real. En XGBoost las predicciones cubren un rango un poco mayor, pero los retrasos extremos todavía se subestiman.



(a) Retraso predicho contra retraso real.



(b) Error según el retraso real.

Figura 2.2: Diagnóstico de XGBoost sobre el conjunto de prueba.

2.4.1. Conclusión sobre sesgo, varianza y ajuste

XGBoost tiene un **sesgo alto**, aunque es menor que el del modelo manual. Su R^2 en *test* es de 0.0479, por lo que explica aproximadamente 4.8 % de los cambios en los

retrasos. También tiene una **varianza baja**, pues los resultados de *train*, *validation* y *test* son muy parecidos. Por lo tanto, este modelo también tiene **underfitting**. Aunque funciona mejor que el modelo manual, todavía es demasiado limitado para explicar los retrasos extremos con las variables disponibles. No se observa *overfitting*, ya que su desempeño no empeora de forma importante en *validation* o *test*.

El error medio de la tabla fue de +0,0567 minutos en *test*. Esto significa que, en promedio, el modelo casi no sobreestima ni subestima los retrasos. Sin embargo, esto no quiere decir que el sesgo del modelo sea bajo. El R^2 continúa siendo pequeño y las gráficas muestran que los retrasos muy grandes normalmente se subestiman.

2.5. Comparación y veredicto final

Al comparar ambos modelos, XGBoost disminuye el MAE en 2.31 %, el RMSE en 1.53 % y el error absoluto mediano en 6.76 %. También aumenta R^2 de 0.0181 a 0.0479. En los márgenes de ± 5 y ± 10 minutos, mejora 3.79 y 3.09 puntos porcentuales. El modelo manual obtiene 1.58 puntos más dentro de ± 15 minutos, mientras que dentro de ± 30 minutos prácticamente empatan. Aunque la diferencia no es muy grande, XGBoost obtiene mejores resultados en la mayoría de las métricas y explica una mayor parte de los retrasos. Por estas razones, seleccioné XGBoost como el modelo final construido con *framework*.

La ejecución completa tardó 5 minutos con 41 segundos en mi computadora. Este tiempo incluye el ETL, la generación de las gráficas, el entrenamiento del modelo manual, el entrenamiento de XGBoost y la evaluación final.

Por último, en la figura 2.3 agrupé los vuelos para comparar el promedio real con el promedio predicho. Esta gráfica sirve para observar la tendencia general sin mostrar

Modelo	MAE	RMSE	R^2	MedAE	± 5	± 10	± 15	± 30	Error medio
Manual	14.0481	29.1341	0.0181	9.1173	23.74 %	55.41 %	79.69 %	92.93 %	-0,2175
XGBoost	13.7230	28.6896	0.0479	8.5006	27.53 %	58.50 %	78.11 %	92.94 %	0.0567

Cuadro 2.3: Comparación final de los modelos sobre el conjunto de prueba.

muchos puntos al mismo tiempo. Mientras más cerca se encuentre cada línea de la línea verde, mejor sigue el modelo los retrasos reales.

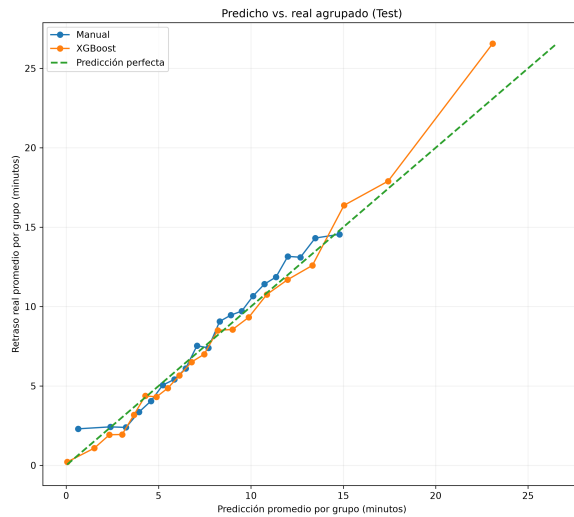


Figura 2.3: Promedios predichos y reales en *test*. XGBoost se acerca más a los valores reales, aunque ninguno de los dos modelos logra representar bien los valores extremos.

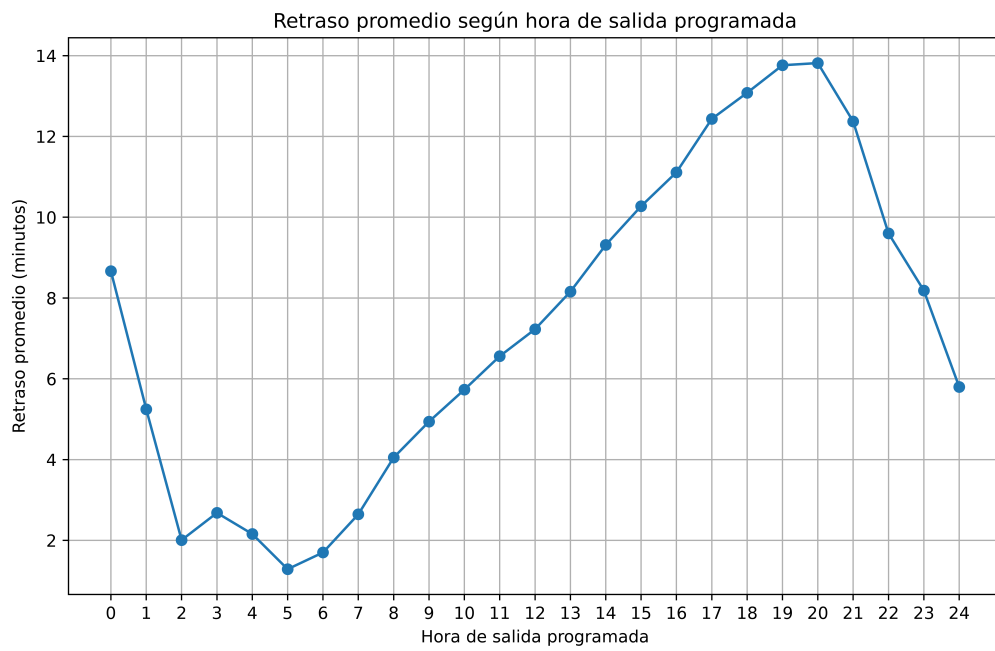
Bibliografía

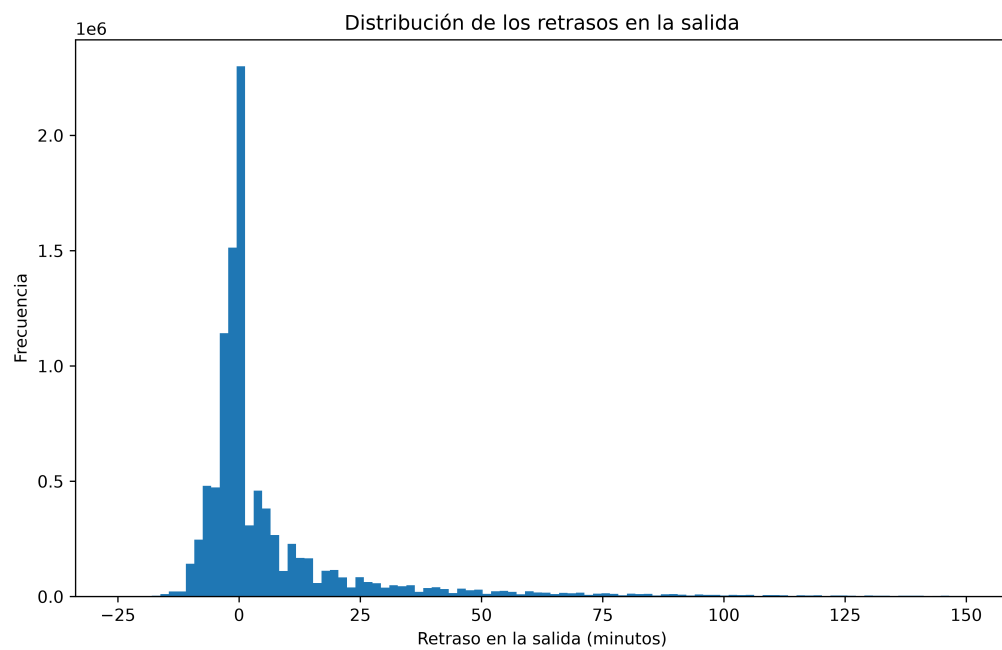
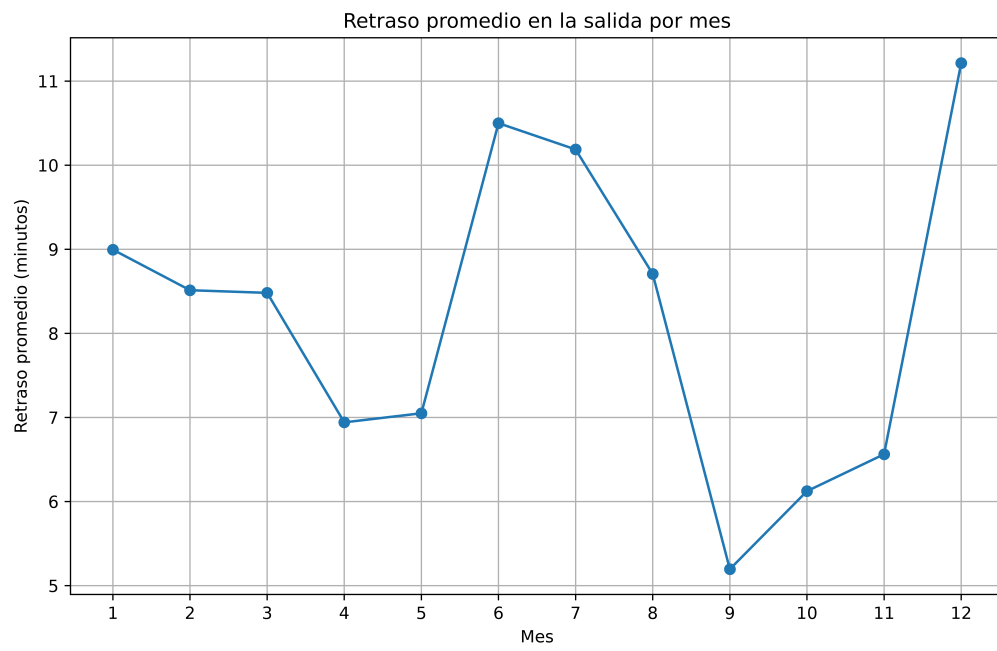
- [1] Matthew Kosinski Annie Badman. *¿Qué es Apache Parquet? / IBM* — *ibm.com*. <https://www.ibm.com/mx-es/think/topics/parquet>. [Accessed 26-08-2026].
- [2] Zhen Guo et al. “A Novel Hybrid Method for Flight Departure Delay Prediction Using Random Forest Regression and Maximal Information Coefficient”. En: *Aerospace Science and Technology* 116 (2021), pág. 106822. DOI: [10.1016/j.ast.2021.106822](https://doi.org/10.1016/j.ast.2021.106822).
- [3] Anish M. Kalliguddi y Aera K. Leboulluec. “Predictive Modeling of Aircraft Flight Delay”. En: *Universal Journal of Management* 5.10 (2017), págs. 485-491. DOI: [10.13189/ujm.2017.051003](https://doi.org/10.13189/ujm.2017.051003).
- [4] Qiang Li y Ranzhe Jing. “Generation and Prediction of Flight Delays in Air Transport”. En: *IET Intelligent Transport Systems* 15.6 (2021), págs. 740-753. DOI: [10.1049/itr2.12057](https://doi.org/10.1049/itr2.12057).
- [5] Suvojit Manna et al. “A Statistical Approach to Predict Flight Delay Using Gradient Boosted Decision Tree”. En: *2017 International Conference on Computational Intelligence in Data Science (ICCIDS)*. 2017. DOI: [10.1109/ICCIDS.2017.8272656](https://doi.org/10.1109/ICCIDS.2017.8272656).
- [6] Juan Jose Rebollo y Hamsa Balakrishnan. “Characterization and Prediction of Air Traffic Delays”. En: *Transportation Research Part C: Emerging Technologies* 44 (2014), págs. 231-241. DOI: [10.1016/j.trc.2014.04.007](https://doi.org/10.1016/j.trc.2014.04.007).
- [7] scgupta. *Training, Validation, and Test Split for ML Models* — *ml4devs.com*. <https://www.ml4devs.com/what-is/train-validation-test-data-split/>. [Accessed 26-08-2026].

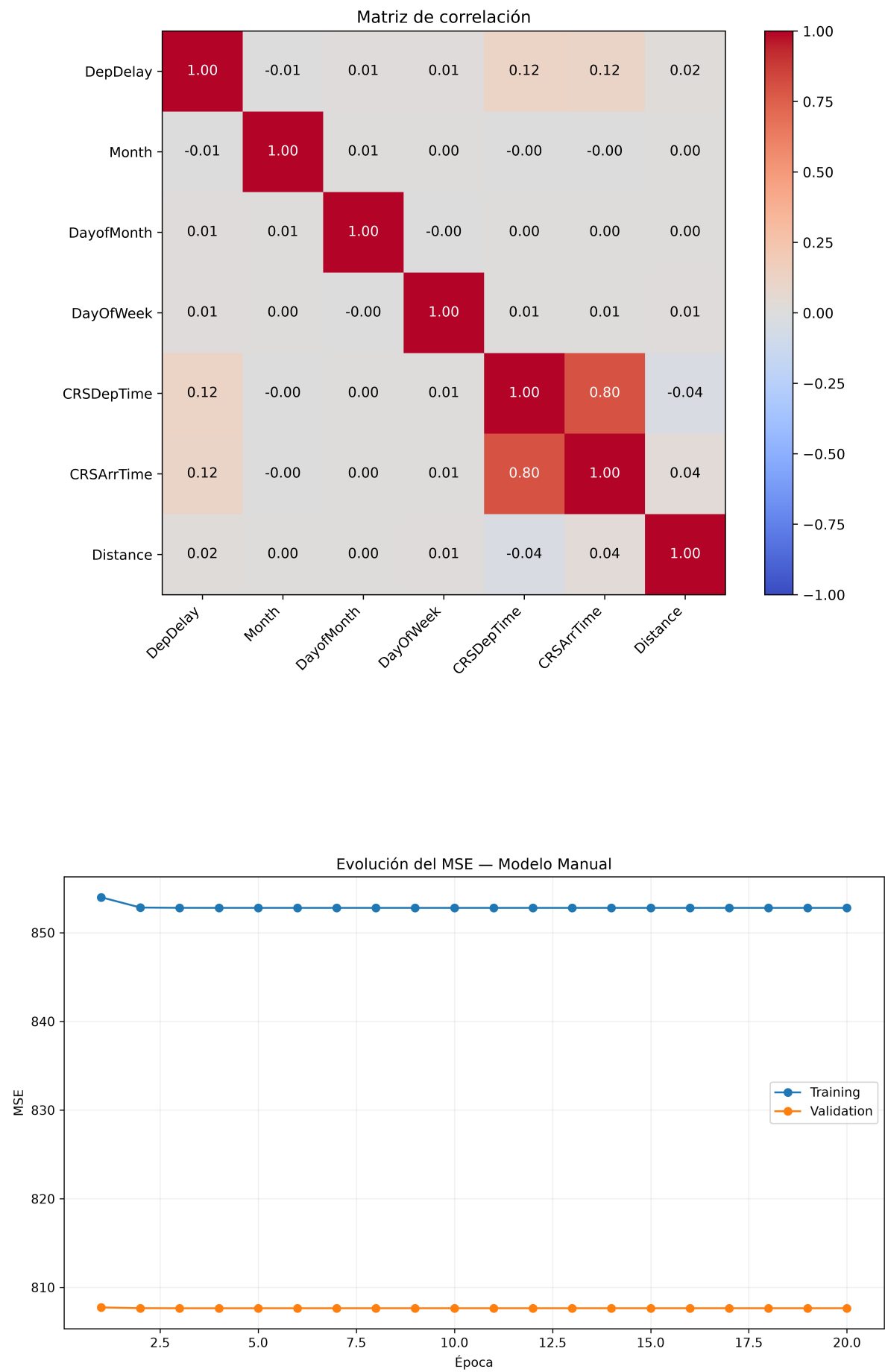
Apéndice A

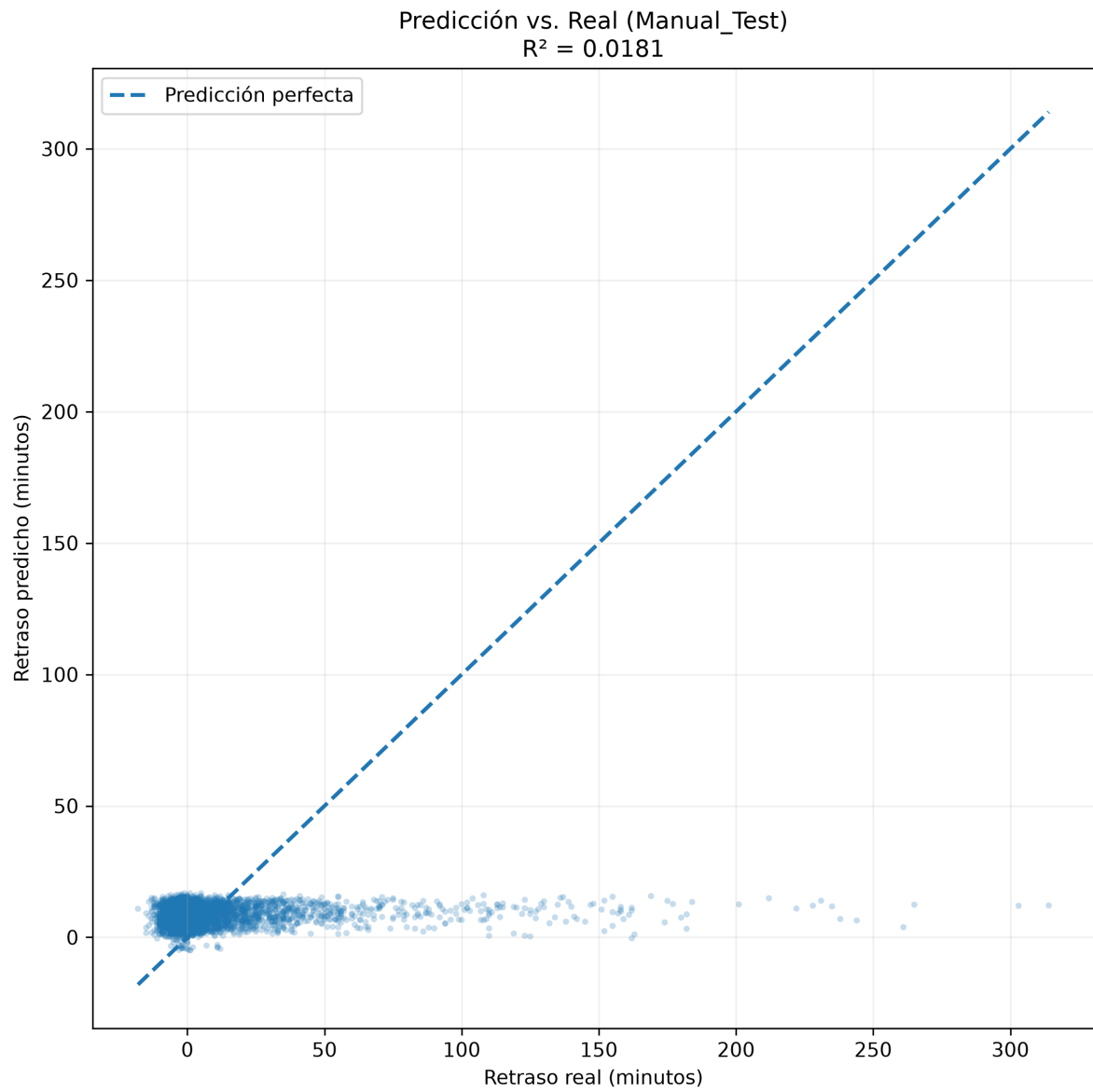
Figuras ampliadas

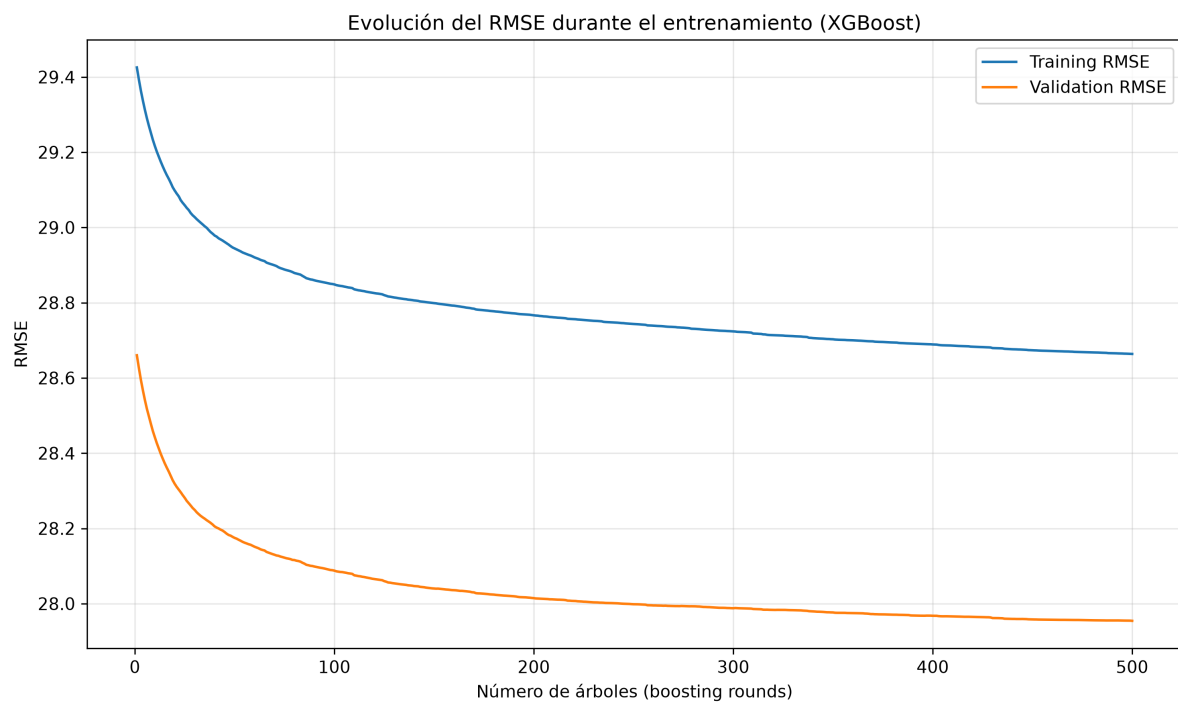
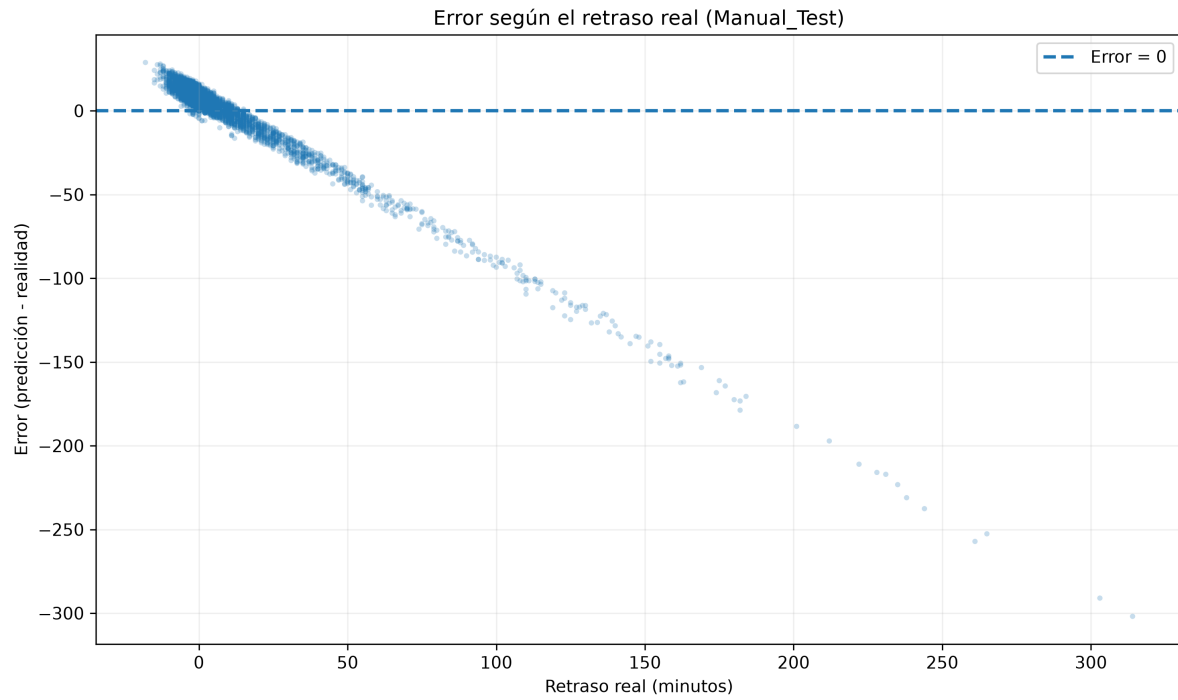


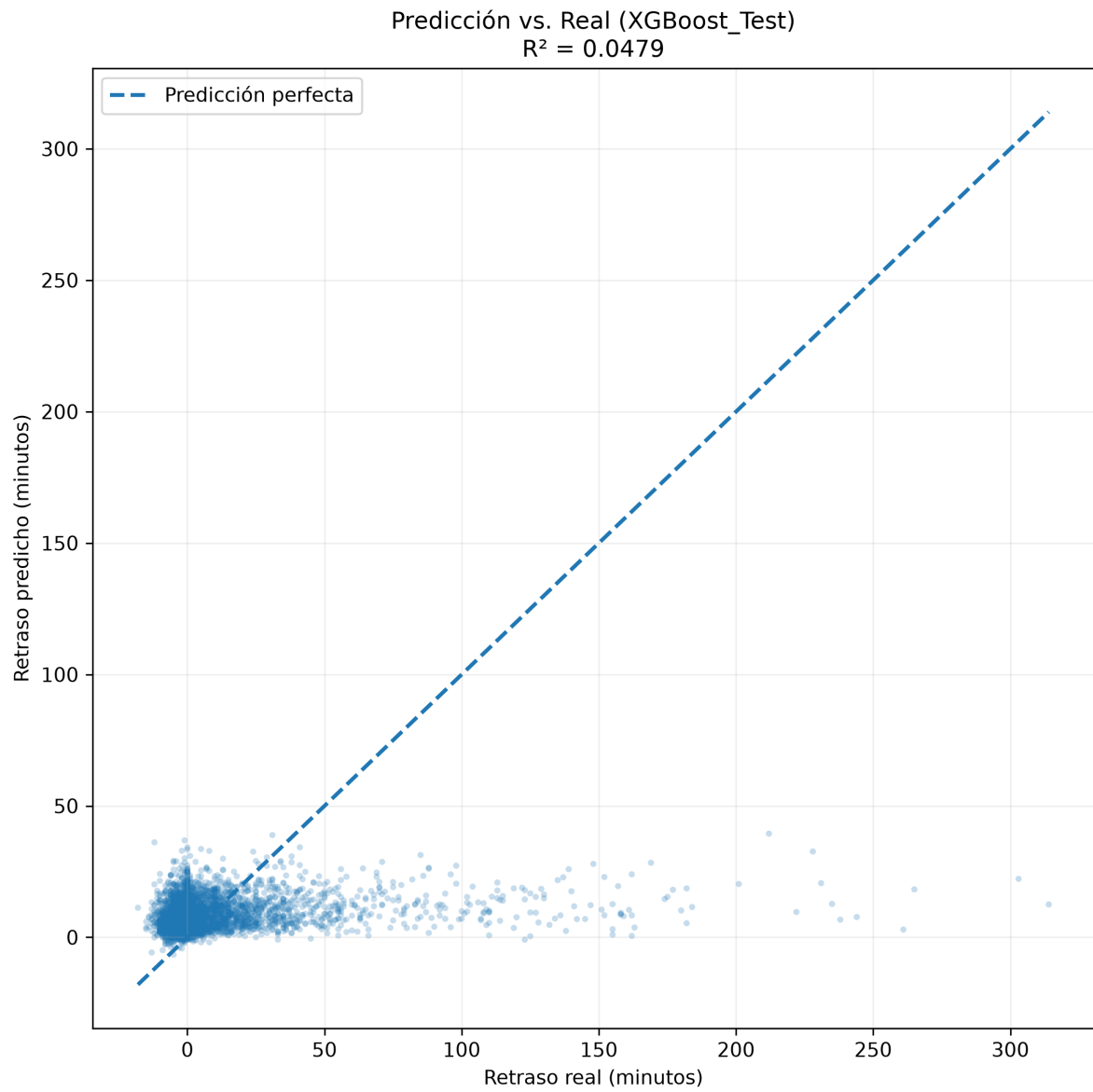


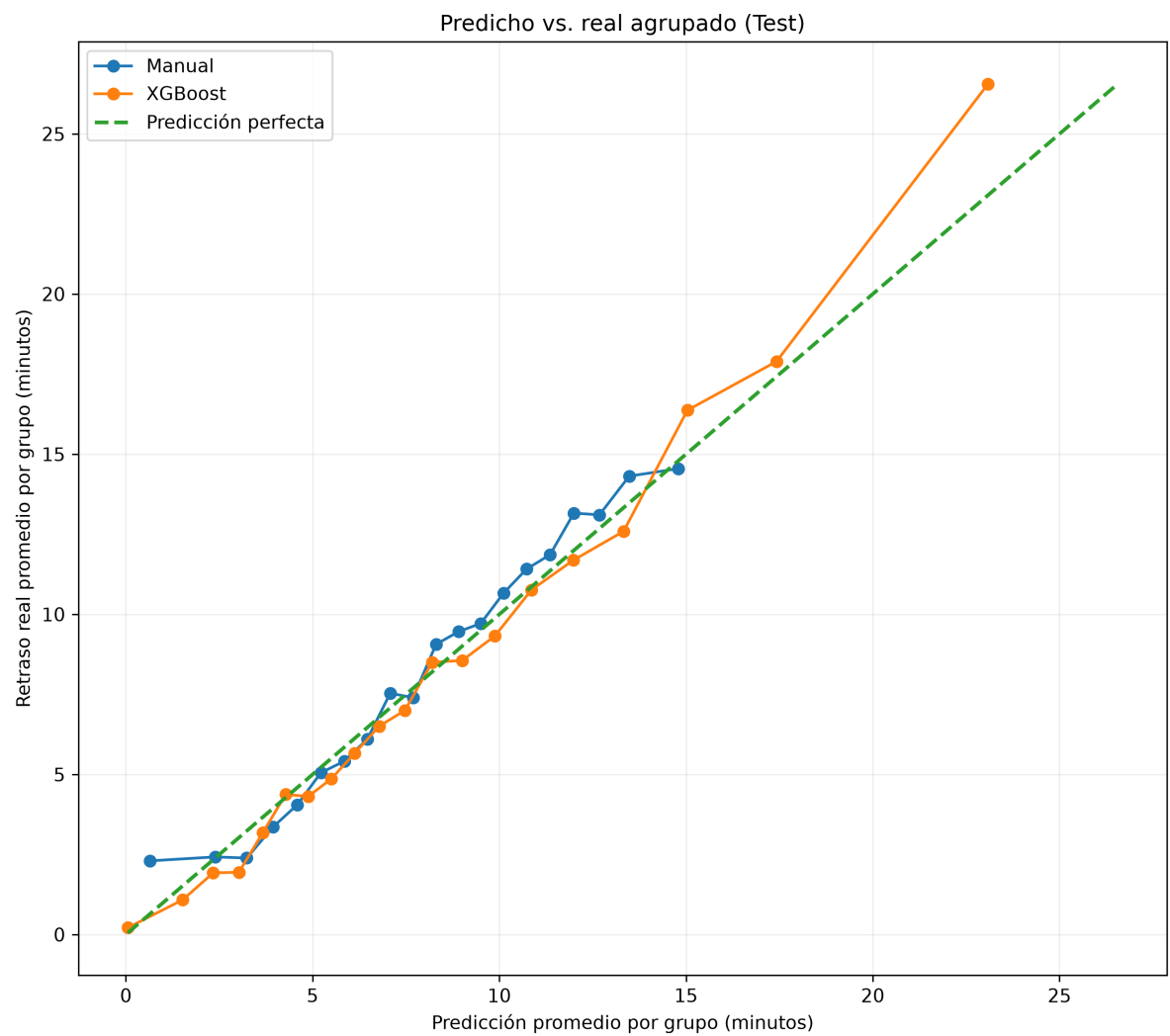
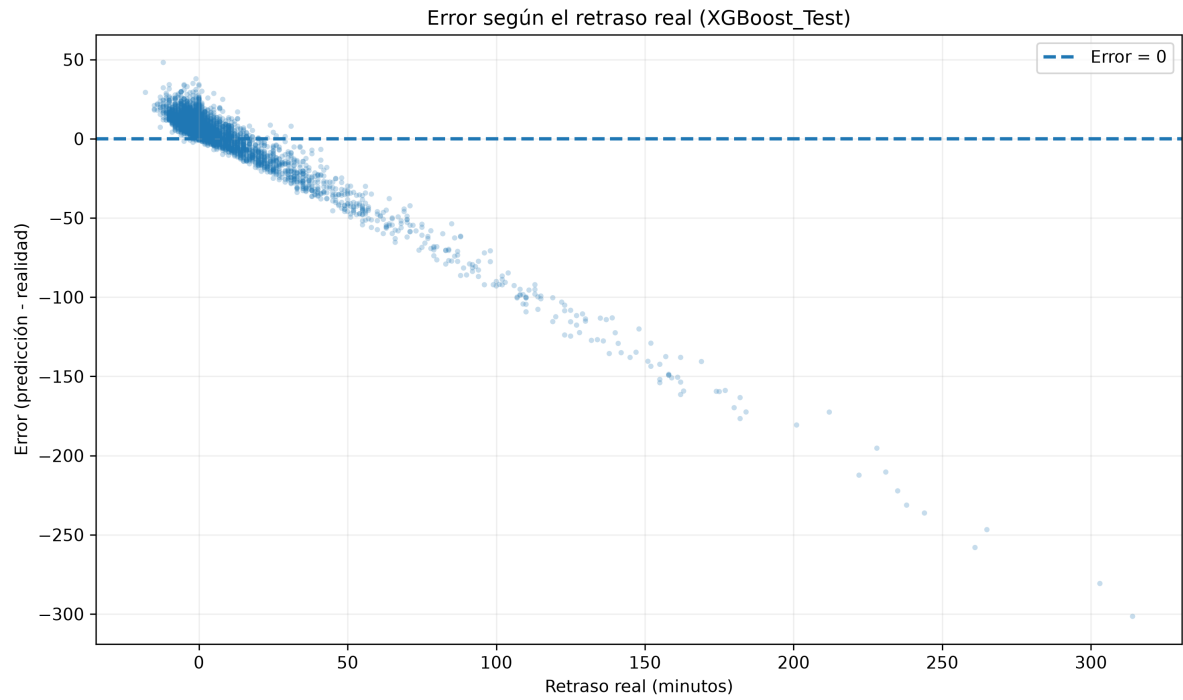












Apéndice B

Códigos

El siguiente código sirve para generar las gráficas del proyecto, aunque algunas funciones no se utilizaron en la versión final del reporte.

```
1 import os
2 import matplotlib.pyplot as plt
3 import numpy as np
4 import pandas as pd
5 import seaborn as sns
6
7 DPI = 300
8
9 def ensure_graphics_directory():
10     os.makedirs("./graficas", exist_ok=True)
11
12 def correlation_matrix(df: pd.DataFrame):
13     candidate_columns = [
14         "DepDelay",
15         "CRSDepTime",
16         "CRSArrTime",
17         "Distance",
18         "Month_sin",
19         "Month_cos",
20         "DayofMonth_sin",
21         "DayofMonth_cos",
22         "DayOfWeek_sin",
23         "DayOfWeek_cos",
24     ]
25     columns = [column for column in candidate_columns if column in
26                 df.columns]
27     ensure_graphics_directory()
28     corr = df[columns].corr()
29     plt.figure(figsize=(9, 7))
30     plt.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
31     plt.colorbar()
32     plt.xticks(range(len(columns)), columns, rotation=45, ha="
33                 right")
34     plt.yticks(range(len(columns)), columns)
35     for i in range(len(columns)):
```

```

34         for j in range(len(columns)):
35             plt.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center",
                        va="center", color="white" if abs(corr.iloc[i, j])
                        > 0.6 else "black")
36     plt.title("Matriz de correlación")
37     plt.tight_layout()
38     plt.savefig("./graficas/correlation_matrix.png", dpi=DPI)
39     plt.close()
40
41
42 def delays_hist(df: pd.DataFrame):
43     ensure_graphics_directory()
44     plt.figure(figsize=(10, 6))
45     bins = int(np.sqrt(len(df)))
46     plt.hist(df["DepDelay"], bins=100, range=(-25, 150))
47     plt.xlabel("Retraso en la salida (minutos)")
48     plt.ylabel("Frecuencia")
49     plt.title("Distribución de los retrasos en la salida")
50     plt.savefig("./graficas/depdelay_histogram.png", dpi=DPI)
51     plt.close()
52
53
54 def mean_delay_by_month(df: pd.DataFrame):
55     ensure_graphics_directory()
56     month = _temporal_values(df, "Month", 12)
57     monthly_mean = df.assign(Month=month).groupby("Month")["
        DepDelay"].mean()
58     plt.figure(figsize=(10, 6))
59     plt.plot(monthly_mean.index, monthly_mean.values, marker="o")
60     plt.xlabel("Mes")
61     plt.ylabel("Retraso promedio (minutos)")
62     plt.title("Retraso promedio en la salida por mes")
63     plt.xticks(range(1, 13))
64     plt.grid()
65     plt.savefig("./graficas/mean_depdelay_by_month.png", dpi=DPI)
66     plt.close()
67
68
69 def mean_delay_by_week(df: pd.DataFrame):
70     ensure_graphics_directory()
71     day_of_week = _temporal_values(df, "DayOfWeek", 7)
72     weekly_mean = df.assign(DayOfWeek=day_of_week).groupby("
        DayOfWeek")["DepDelay"].mean()
73     plt.figure(figsize=(10, 6))
74     plt.plot(weekly_mean.index, weekly_mean.values, marker="o")
75     plt.xlabel("Día de la semana")
76     plt.ylabel("Retraso promedio (minutos)")
77     plt.title("Retraso promedio en la salida por día de la semana"
78             )
79     plt.xticks(range(1, 8))
80     plt.grid()

```

```

80     plt.savefig("./graficas/mean_depdelay_by_weekly.png", dpi=DPI)
81     plt.close()
82
83
84 def mean_delay_by_hour(df: pd.DataFrame):
85     ensure_graphics_directory()
86     temp = df.copy()
87     temp["Hour"] = (temp["CRSDepTime"] // 60).astype(int)
88     hourly_mean = temp.groupby("Hour")["DepDelay"].mean()
89     plt.figure(figsize=(10, 6))
90     plt.plot(hourly_mean.index, hourly_mean.values, marker="o")
91     plt.xlabel("Hora de salida programada")
92     plt.ylabel("Retraso promedio (minutos)")
93     plt.title("Retraso promedio según hora de salida programada")
94     plt.xticks(range(25))
95     plt.grid()
96     plt.savefig("./graficas/mean_depdelay_by_hour.png", dpi=DPI)
97     plt.close()
98
99
100 def _temporal_values(df: pd.DataFrame, column: str, period: int):
101     """Recupera la categoría temporal desde su par seno/coseno si
102         hace falta."""
103     if column in df.columns:
104         return df[column]
105
106     sin_column = f"{column}_sin"
107     cos_column = f"{column}_cos"
108     if sin_column not in df.columns or cos_column not in df.
109         columns:
110         raise ValueError(
111             f"No se encontró {column} ni el par {sin_column}/{
112                 cos_column}."
113         )
114
115     angles = np.mod(np.arctan2(df[sin_column], df[cos_column]), 2
116         * np.pi)
117     values = np rint(angles * period / (2 * np.pi)).astype(int)
118     return values.where(values != 0, period)
119
120
121 def pair_plot(df: pd.DataFrame):
122     print(df)
123     df = df.iloc[:, 0:7]
124     sns.pairplot(df, hue="DayOfWeek").savefig("./graficas/
125         pair_plot.png", dpi=DPI)
126
127 def predicted_vs_actual(y_real, y_pred, r2, dataset_name="Test",
128     max_points=5000):
129     """
130     Predicción vs realidad.

```

```
125     Si hay demasiadas observaciones se utiliza sampling
126     para evitar intentar dibujar millones de puntos.
127     """
```

```
128
129     ensure_graphics_directory()
```

```
130
131     y_real = np.asarray(y_real)
132     y_pred = np.asarray(y_pred)
```

```
133
134     if len(y_real) > max_points:
```

```
135
136         rng = np.random.default_rng(42)
```

```
137
138         indices = rng.choice(
139             len(y_real),
140             size=max_points,
141             replace=False
142         )
```

```
143
144         y_real_plot = y_real[indices]
145         y_pred_plot = y_pred[indices]
```

```
146
147     else:
```

```
148
149         y_real_plot = y_real
150         y_pred_plot = y_pred
```

```
151
152     min_value = min(
153         y_real_plot.min(),
154         y_pred_plot.min()
155     )
```

```
156     max_value = max(
157         y_real_plot.max(),
158         y_pred_plot.max()
159     )
```

```
160
161     plt.figure(figsize=(8, 8))
```

```
162     plt.scatter(
163         y_real_plot,
164         y_pred_plot,
165         s=10,
166         alpha=0.25,
167         color="#1f77b4",
168         edgecolors="none"
169     )
```

```
170     plt.plot(
171         [min_value, max_value],
172         [min_value, max_value],
173         "--",
174         linewidth=2,
175         label="Predicción perfecta"
```

```

176     )
177     plt.xlabel("Retraso real (minutos)")
178     plt.ylabel("Retraso predicho (minutos)")
179     plt.title(
180         f"Predicción vs. Real ({dataset_name})\n"
181         f"R = {r2:.4f}"
182     )
183     plt.legend()
184     plt.grid(alpha=0.2)
185     plt.tight_layout()
186     plt.savefig(
187         f"./graficas/predicted_vs_actual_{dataset_name.lower()}.png",
188         dpi=DPI
189     )
190     plt.close()
191
192
193 def cost_evolution(train_costs, validation_costs, train_r2,
194                   validation_r2, test_r2):
195     plt.figure(figsize=(10, 6))
196     plt.plot(range(1, len(train_costs) + 1), train_costs, label="
197             Training")
198     plt.plot(range(1, len(validation_costs) + 1), validation_costs
199             , label="Validation")
200     plt.xlabel("Época")
201     plt.ylabel("MSE")
202     plt.title("Evolución del error durante el entrenamiento")
203     r2_text = (f"$R^2$ train: {train_r2:.4f}\n" f"$R^2$ validation
204             : {validation_r2:.4f}\n" f"$R^2$ test: {test_r2:.4f}")
205     plt.gca().text(
206         0.98, 0.95, r2_text,
207         transform=plt.gca().transAxes,
208         ha="right", va="top",
209         fontsize=10,
210         bbox=dict(boxstyle="round", facecolor="white", alpha=0.8)
211     )
212     plt.legend()
213     plt.grid()
214     plt.tight_layout()
215     plt.savefig("./cost_evolution.png", dpi=DPI)
216
217
218 def rf_cost_evolution(train_mse: list, validation_mse: list,
219                       estimators: list):
220     plt.figure(figsize=(10, 6))
221     plt.plot(estimators, train_mse, marker="o", label="Training MSE")
222     plt.plot(estimators, validation_mse, marker="o", label="
223             Validation MSE")
224     plt.xlabel("Número de árboles")
225     plt.ylabel("MSE")
226     plt.title("Evolución del MSE según el número de árboles")

```

```

220     plt.legend()
221     plt.grid(alpha=0.3)
222     plt.tight_layout()
223     plt.savefig("./graficas/rf_cost_evolution.png", dpi=DPI)
224     plt.close()
225
226
227 def xgb_cost_evolution(train_rmse: list, val_rmse: list):
228     """
229     Grafica la evolución del RMSE de train y validation por cada á
230     rbol
231     (boosting round) que XGBoost va agregando al modelo.
232     """
233     plt.figure(figsize=(10, 6))
234     iterations = range(1, len(train_rmse) + 1)
235     plt.plot(iterations, train_rmse, label="Training RMSE")
236     plt.plot(iterations, val_rmse, label="Validation RMSE")
237
238     plt.xlabel("Número de árboles (boosting rounds)")
239     plt.ylabel("RMSE")
240     plt.title("Evolución del RMSE durante el entrenamiento (
241             XGBoost)")
242     plt.legend()
243     plt.grid(alpha=0.3)
244     plt.tight_layout()
245     plt.savefig("./graficas/xgb_cost_evolution.png", dpi=DPI)
246     plt.close()
247
248
249 def residual_plot(y_real, y_pred, dataset_name="Test", max_points
250 =5000):
251     """
252     Muestra el error (predicción - realidad) según el retraso real
253     .
254     """
255     ensure_graphics_directory()
256
257     y_real = np.asarray(y_real)
258     y_pred = np.asarray(y_pred)
259     residuals = y_pred - y_real
260
261     if len(y_pred) > max_points:
262         rng = np.random.default_rng(42)
263         indices = rng.choice(
264             len(y_pred),
265             size=max_points,
266             replace=False
267         )
268         y_plot = y_real[indices]
269         residuals_plot = residuals[indices]
270     else:

```

```
267     y_plot = y_real
268     residuals_plot = residuals
269
270     plt.figure(figsize=(10, 6))
271     plt.scatter(
272         y_plot,
273         residuals_plot,
274         s=10,
275         alpha=0.25,
276         color="#1f77b4",
277         edgecolors="none"
278     )
279     plt.axhline(
280         0,
281         linestyle="--",
282         linewidth=2,
283         label="Error = 0"
284     )
285     plt.xlabel("Retraso real (minutos)")
286     plt.ylabel("Error (predicción - realidad)")
287     plt.title(
288         f"Error según el retraso real ({dataset_name})"
289     )
290     plt.legend()
291     plt.grid(alpha=0.2)
292     plt.tight_layout()
293     plt.savefig(
294         f"./graficas/residuals_{dataset_name.lower()}.png",
295         dpi=DPI
296     )
297     plt.close()
298
299
300 def error_distribution(y_real, y_pred, dataset_name="Test"):
301     ensure_graphics_directory()
302
303     y_real = np.asarray(y_real)
304     y_pred = np.asarray(y_pred)
305     errors = y_pred - y_real
306
307     plt.figure(figsize=(10, 6))
308     plt.hist(
309         errors,
310         bins=100
311     )
312     plt.axvline(
313         0,
314         linestyle="--",
315         linewidth=2
316     )
317     plt.xlabel("Error (predicción - realidad)")
```

```
318 plt.ylabel("Frecuencia")
319 plt.title(
320     f"Distribución de errores ({dataset_name})"
321 )
322 plt.grid(alpha=0.2)
323 plt.tight_layout()
324 plt.savefig(
325     f"./graficas/error_distribution_{dataset_name.lower()}.png",
326     dpi=DPI
327 )
328 plt.close()
329
330
331 def binned_prediction_plot(
332     models_predictions,
333     y_real,
334     n_bins=20,
335     dataset_name="Test"
336 ):
337     """
338     Agrupa las predicciones en cuantiles y compara:
339
340         promedio predicho
341         vs
342         promedio real
343
344     Permite comparar varios modelos en una misma gráfica.
345     """
346
347     ensure_graphics_directory()
348
349     y_real = np.asarray(y_real)
350
351     plt.figure(figsize=(9, 8))
352
353     min_value = float("-inf")
354     max_value = float("inf")
355
356     for model_name, predictions in models_predictions.items():
357
358         y_pred = np.asarray(predictions)
359
360         order = np.argsort(y_pred)
361
362         sorted_pred = y_pred[order]
363         sorted_real = y_real[order]
364
365         bins = np.array_split(
366             np.arange(len(sorted_pred)),
367             n_bins
```

```
368     )
369
370     pred_means = []
371     real_means = []
372
373     for indices in bins:
374
375         pred_means.append(
376             sorted_pred[indices].mean()
377         )
378
379         real_means.append(
380             sorted_real[indices].mean()
381         )
382
383     min_value = min(
384         min_value,
385         min(pred_means),
386         min(real_means)
387     )
388
389     max_value = max(
390         max_value,
391         max(pred_means),
392         max(real_means)
393     )
394
395     plt.plot(
396         pred_means,
397         real_means,
398         marker="o",
399         label=model_name
400     )
401
402     plt.plot(
403         [min_value, max_value],
404         [min_value, max_value],
405         "--",
406         linewidth=2,
407         label="Predicción perfecta"
408     )
409
410     plt.xlabel(
411         "Predicción promedio por grupo (minutos)"
412     )
413
414     plt.ylabel(
415         "Retraso real promedio por grupo (minutos)"
416     )
417
418     plt.title(
```

```
419         f"Predicho vs. real agrupado ({dataset_name})"
420     )
421
422     plt.legend()
423
424     plt.grid(alpha=0.2)
425
426     plt.tight_layout()
427
428     plt.savefig(
429         f"./graficas/binned_prediction_{dataset_name.lower()}.png"
430         ,
431         dpi=DPI
432     )
433
434     plt.close()
435
436 def compare_models_bar(
437     results,
438     metric,
439     title=None
440 ):
441     """
442     Compara modelos mediante una gráfica de barras.
443
444     results:
445         {
446             "Random Forest": metrics,
447             "XGBoost": metrics
448         }
449
450     metric:
451         "test_mae"
452         "test_rmse"
453         "test_r2"
454         etc.
455     """
456
457     ensure_graphics_directory()
458
459     model_names = list(results.keys())
460
461     values = [
462         results[name][metric]
463         for name in model_names
464     ]
465
466     plt.figure(figsize=(9, 6))
467
468     bars = plt.bar(
```

```
469     model_names ,
470     values
471 )
472
473 for bar, value in zip(bars, values):
474
475     plt.text(
476         bar.get_x() + bar.get_width() / 2,
477         bar.get_height(),
478         f"{value:.3f}",
479         ha="center",
480         va="bottom"
481     )
482
483 plt.ylabel(metric)
484
485 if title is None:
486     title = f"Comparación      {metric}"
487
488 plt.title(title)
489
490 plt.xticks(
491     rotation=15
492 )
493
494 plt.grid(
495     axis="y",
496     alpha=0.2
497 )
498
499 plt.tight_layout()
500
501 plt.savefig(
502     f"./graficas/comparison_{metric}.png",
503     dpi=DPI
504 )
505
506 plt.close()
507
508
509 def plot_training_history(
510     x,
511     train_values,
512     validation_values,
513     xlabel,
514     ylabel,
515     title,
516     filename
517 ):
518     """
519     Gráfica genérica de aprendizaje.
```

```
520     """
521
522     ensure_graphics_directory()
523
524     plt.figure(figsize=(10, 6))
525
526     plt.plot(
527         x,
528         train_values,
529         marker="o",
530         label="Training"
531     )
532
533     plt.plot(
534         x,
535         validation_values,
536         marker="o",
537         label="Validation"
538     )
539
540     plt.xlabel(xlabel)
541     plt.ylabel(ylabel)
542
543     plt.title(title)
544
545     plt.legend()
546
547     plt.grid(alpha=0.2)
548
549     plt.tight_layout()
550
551     plt.savefig(
552         f"./graficas/{filename}.png",
553         dpi=DPI
554     )
555
556     plt.close()
```

Proceso de ETL.

```
1  import os
2  import pandas as pd
3  import numpy as np
4  from scipy.io import arff
5
6
7  RAW_DATA_PATH = "./airlines_train_regression_10000000.arff"
8  PARQUET_PATH = "airlines_10M.parquet"
9
10 TRAIN_FRACTION = 0.98
11 VAL_FRACTION = 0.01
12 TEST_FRACTION = 0.01
```

```
13 RANDOM_STATE = 42
14
15
16 def hhmm_to_minutes(time):
17     time = int(time)
18     if time == 2400:
19         return 0
20
21     hours = time // 100
22     minutes = time % 100
23
24     if not 0 <= hours <= 23 or not 0 <= minutes <= 59:
25         raise ValueError(f"Hora HHMM inválida: {time}")
26
27     return hours * 60 + minutes
28
29
30 def _valid_hhmm_mask(series: pd.Series):
31     values = pd.to_numeric(series, errors="coerce")
32     hours = values // 100
33     minutes = values % 100
34
35     is_integer = values.notna() & np.isfinite(values) & (values ==
36         np.floor(values))
37     is_regular_time = (
38         values.between(0, 2359)
39         & hours.between(0, 23)
40         & minutes.between(0, 59)
41     )
42
43     return is_integer & (is_regular_time | values.eq(2400))
44
45
46 def _remove_invalid_scheduled_times(df: pd.DataFrame):
47     time_columns = ("CRSDepTime", "CRSArrTime")
48     missing_columns = [column for column in time_columns if column
49         not in df.columns]
50     if missing_columns:
51         raise ValueError(
52             "Faltan columnas de horario requeridas: " + ", ".join(
53                 missing_columns)
54         )
55
56     valid_rows = pd.Series(True, index=df.index)
57     for column in time_columns:
58         valid_rows &= _valid_hhmm_mask(df[column])
59
60     removed_instances = int((~valid_rows).sum())
61     return df.loc[valid_rows].copy(), removed_instances
```

```
61 def _decode_categorical(value):
62     if isinstance(value, bytes):
63         return value.decode("utf-8")
64     return str(value)
65
66 def extract_data():
67     print("Cargando los datos...")
68
69     if os.path.exists(PARQUET_PATH):
70         df = pd.read_parquet(PARQUET_PATH, engine="pyarrow")
71         for time_column in ("CRSDepTime", "CRSArrTime"):
72             if time_column in df.columns:
73                 df.loc[df[time_column] == 1440, time_column] = 0
74         print("\nDatos generales:")
75         print(df.info())
76         print("\nDatos ya transformados. Saltando transformación."
77             )
78
79         return df, True
80
81     arff_data = arff.loadarff(RAW_DATA_PATH)
82     df = pd.DataFrame(arff_data[0])
83     print("\nEstado inicial del dataset:")
84     print(df.info())
85
86     return df, False
87
88 def transform_data(df: pd.DataFrame):
89     print("\nConvirtiendo CRSDepTime y CRSArrTime a minutos desde
90         la medianoche...")
91
92     df, removed_instances = _remove_invalid_scheduled_times(df)
93
94     df["CRSDepTime"] = (df["CRSDepTime"].apply(hhmm_to_minutes))
95     df["CRSArrTime"] = (df["CRSArrTime"].apply(hhmm_to_minutes))
96
97     categorical_columns = [
98         "UniqueCarrier",
99         "Origin",
100         "Dest"
101     ]
102
103     cyclical_features = {
104         "Month": 12.0,
105         "DayofMonth": 31.0,
106         "DayOfWeek": 7.0
107     }
108
109     for col, max_val in cyclical_features.items():
110         if col in df.columns:
```

```

110         df[col] = df[col].astype(float)
111         df[f"{col}_sin"] = np.sin(2 * np.pi * df[col] /
112             max_val)
113         df[f"{col}_cos"] = np.cos(2 * np.pi * df[col] /
114             max_val)
115         df.drop(columns=[col], inplace=True)
116
117     for column in categorical_columns:
118         if column in df.columns:
119             df[column] = (df[column].apply(_decode_categorical))
120
121     print("Conversión realizada.")
122     print("\nDatos generales:")
123     print(df.info())
124     print("\nEstadísticas:")
125     print(df.describe())
126     print("\nCantidad de valores nulos:")
127     print(df.isna().sum())
128     print("\nVariables categóricas conservadas:")
129     print(categorical_columns)
130
131     print("\nGuardando datos limpios en parquet...")
132     df.to_parquet(PARQUET_PATH, engine="pyarrow")
133     print("Datos en parquet guardados.")
134     print(
135         "Total de instancias eliminadas por horarios inválidos: "
136         f"{removed_instances:,}"
137     )
138
139     return df
140
141 def split_data(df: pd.DataFrame, train_fraction=TRAIN_FRACTION,
142     val_fraction=VAL_FRACTION,
143     test_fraction=TEST_FRACTION, shuffle=True,
144     random_state=RANDOM_STATE):
145     total_fraction = (train_fraction + val_fraction +
146         test_fraction)
147     if abs(total_fraction - 1.0) > 1e-9:
148         raise ValueError("Las proporciones de split deben sumar 1.")
149
150     if len(df) == 0:
151         raise ValueError("No se puede dividir un DataFrame vacío.")
152
153     n = len(df)
154     train_end = int(n * train_fraction)
155     val_end = train_end + int(n * val_fraction)
156
157     if shuffle:

```

```
154     rng = np.random.default_rng(random_state)
155     positions = rng.permutation(n)
156     else:
157         positions = np.arange(n)
158
159     train_df = df.iloc[positions[:train_end]].copy()
160     val_df = df.iloc[positions[train_end:val_end]].copy()
161     test_df = df.iloc[positions[val_end:]].copy()
162
163     return df, train_df, val_df, test_df
```

Implementación del modelo de regresión lineal.

```
1 import math
2 import random
3 from tqdm import tqdm # Para crear una barra de progreso al
   momento de entrenar el modelo
4
5 def hypo(params: list, weights: list, bias: float) -> float:
6     result = 0
7     for i in range(len(params)):
8         result += params[i] * weights[i]
9     return result + bias
10
11 def error(y_pred:float,y_real:float)->float:
12     return y_pred - y_real
13
14 def cost(y_hat:list, y:list)->float:
15     result = 0
16     for i in range(len(y)):
17         result += (error(y_hat[i],y[i]))**2
18     result = 1/len(y) * result
19     return result
20
21 def r_squared(y_pred: list, y_real: list) -> float:
22     n = len(y_real)
23     mean_y = sum(y_real) / n
24     ss_res = 0.0
25     ss_tot = 0.0
26     for i in range(n):
27         ss_res += (y_real[i] - y_pred[i]) ** 2
28         ss_tot += (y_real[i] - mean_y) ** 2
29     return 1 - (ss_res / ss_tot)
30
31 def gradient(weights: list, bias: float, params: list, y: list):
32     gradients = [0] * len(weights)
33     bias_gradient = 0
34     m = len(params)
35
36     for i in range(m):
37         predict = hypo(params[i], weights, bias)
38         predict_error = error(predict, y[i])
```

```
39
40     for j in range(len(weights)):
41         gradients[j] += predict_error * params[i][j]
42         bias_gradient += predict_error
43
44     for j in range(len(weights)):
45         gradients[j] = (2/m) * gradients[j]
46         bias_gradient = (2/m) * bias_gradient
47
48     return gradients, bias_gradient
49
50
51 def update(weights:list, bias:float, gradients:list, bias_gradient
52 :float, lr:float):
53     new_weights = []
54     for i in range(len(weights)):
55         new_weight = weights[i] - (lr * gradients[i])
56         new_weights.append(new_weight)
57     new_bias = bias - (lr * bias_gradient)
58     return new_weights, new_bias
59
60 def predict(params:list, weights:list, bias:float):
61     predictions = []
62     for i in range(len(params)):
63         prediction = hypo(params[i], weights, bias)
64         predictions.append(prediction)
65     return predictions
66
67
68 def train_step(weights:list, bias:float, params:list, y:list, lr:
69 float):
70     predictions = predict(params, weights, bias)
71     current_cost = cost(predictions, y)
72     gradients, bias_gradient = gradient(weights, bias, params, y)
73     new_weights, new_bias = update(weights, bias, gradients,
74                                     bias_gradient, lr)
75     return new_weights, new_bias, current_cost
76
77
78 def train(params:list, y:list, validation_params:list,
79 validation_y:list, lr:float, epochs:int, batch_size:int,
80 random_state:int=42):
81     # weights = [0 for _ in range(len(params[0]))]
82     rng = random.Random(random_state)
83     weights = [rng.random() for _ in range(len(params[0]))]
84     bias = 0
85     train_costs = []
86     validation_costs = []
87
88     m = len(params)
89     num_batches = math.ceil(m / batch_size)
```

```

85
86     total_steps = epochs*num_batches
87     pbar = tqdm(total=total_steps, desc="Entrenando modelo")
88
89     for epoch in range(epochs):
90         epoch_train_cost = 0
91         # Iteramos sobre los datos en pequeños lotes
92         for i in range(num_batches):
93             # Calculamos los índices de inicio y fin
94             start_idx = i * batch_size
95             end_idx = min(start_idx + batch_size, m)
96             # Extraemos el lote actual
97             batch_params = params[start_idx:end_idx]
98             batch_y = y[start_idx:end_idx]
99             # Entrenamos sólo con este lote.
100             weights, bias, batch_cost = train_step(weights, bias,
101                                                     batch_params, batch_y, lr)
102             # Sumamos el costo del lote para sacar el promedio
103             # después
104             epoch_train_cost += batch_cost * len(batch_y)
105             # Actualizar barra de progreso
106             pbar.update(1)
107             pbar.set_postfix({
108                 "ep": f"{epoch+1}/{epochs}",
109                 "batch": f"{i+1}/{num_batches}",
110             })
111
112             # Costo promedio de entrenamiento de toda la época
113             epoch_train_cost /= m
114             train_costs.append(epoch_train_cost)
115
116             # Evaluamos en validación
117             validation_predictions = predict(validation_params,
118                                             weights, bias)
119             validation_cost = cost(validation_predictions,
120                                   validation_y)
121             validation_costs.append(validation_cost)
122
123         pbar.close()
124         return weights, bias, train_costs, validation_costs
125
126 def run_regression_model(X_train,y_train,X_val,y_val,X_test=None,
127                         y_test=None,learning_rate=0.1,epochs=20,batch_size=2048,
128                         random_state=42):
129     params = X_train.values.tolist()
130     train_y = y_train.tolist()
131     validation_params = X_val.values.tolist()
132     validation_y = y_val.tolist()
133     if (X_test is None) != (y_test is None):
134         raise ValueError("X_test y y_test deben proporcionarse

```

```
juntos.")
130
131 weights, bias, train_costs, validation_costs = train(params,
    train_y, validation_params, validation_y, learning_rate,
    epochs, batch_size, random_state)
132
133 train_predictions = predict(params, weights, bias)
134 validation_predictions = predict(validation_params, weights,
    bias)
135 predictions = {
136     "train": train_predictions,
137     "val": validation_predictions,
138     "y_train": train_y,
139     "y_val": validation_y,
140 }
141
142 if X_test is not None:
143     test_params = X_test.values.tolist()
144     test_y = y_test.tolist()
145     predictions["test"] = predict(test_params, weights, bias)
146     predictions["y_test"] = test_y
147
148 history = {
149     "epoch": list(range(1, len(train_costs) + 1)),
150     "train_mse": train_costs,
151     "val_mse": validation_costs,
152     "train_mse_kind": "online_during_updates"
153 }
154
155 model = {
156     "type": "manual_linear_regression",
157     "weights": weights,
158     "bias": bias,
159     "learning_rate": learning_rate,
160     "epochs": epochs,
161     "batch_size": batch_size,
162     "random_state": random_state
163 }
164
165 return model, predictions, history
```

Código principal.

```
1 from experiment import run_experiment
2
3 if __name__ == '__main__':
4     run_experiment()
```